

Model Effectiveness Evaluation Workbench

ELEC5623 Group Business Proposal candidate

Selected direction: **Track B - Product that helps AI work better**

Candidate status: **unapproved, incomplete, and not submission-ready**

Official assessment: 10% (10 marks), group assignment

Due: 8 September 2026, 23:59 through Canvas

Recommended length: 8-10 pages excluding only the cover and references

This compressed candidate follows the official 12-section structure. It does not claim group formation, stakeholder validation, tutor approval, a frozen evaluation result, or Canvas submission. Bracketed fields and **GroupXX** are deliberate blocking sentinels. They must be replaced with verified facts before rebuilding or renaming this candidate as the submission PDF. The group must count every page other than the cover and reference-only pages; a page shared by Section 11 and references remains a body page. No appendix is assumed to be outside the page recommendation. The 8-10 pages are an official recommendation, but the rendered candidate treats that range as its internal page-QA pass condition.

1. Cover and group information

Cover field	Candidate value
Project title	Model Effectiveness Evaluation Workbench
Selected direction	Track B - Product that helps AI work better
Group number	GroupXX - [REQUIRED: actual Canvas group number]
Full member names and student IDs	[REQUIRED: actual confirmed members and SIDs]
Lab tutor	[REQUIRED: authorised lab tutor name]
Approval date	[REQUIRED: actual approval date]
Brief approval statement	[REQUIRED: exact approved wording or verified concise statement]
Product vision	On the same evidence-backed tasks, run ≥ 2 named models; report quality, task-fit, latency, and estimated cost separately; contrast min-cost vs quality/task-fit; human review remains success; not auto model-deploy.

Tutor approval is mandatory before the deadline and must confirm relevance, originality, semester feasibility, and appropriate scope. A passing local test suite is not approval evidence. Suggested final filename, after the group gate is closed: **ELEC5623_GroupXX_Proposal.pdf**.

2. Executive summary

A student can ask two models to write up the same lab notebook. The useful question is which model hallucinates less and fits the stated task better, not whether cheaper means worse. Reviewers still need claim-level evidence, quotations, requirement coverage, and an auditable decision. Candidate users are students, evidence reviewers, requirement owners, authors, and team leads; these roles are hypotheses pending approved validation.

The proposed **Model Effectiveness Evaluation Workbench** runs ≥ 2 named models on one English JSON bundle of requirements, evidence, and AI-generated Markdown. The inner **Inspector** engine (FR-01-15) segments claims, retrieves local evidence, classifies support, enforces citations and exact quotes, maps requirements, and exports an immutable run. The compare shell (FR-16-18) reports quality, task fit, latency, and estimated list-price cost separately and contrasts **min-cost** with a **quality/task-fit** rule. No combined index; no auto model-deploy. A completed run means only **READY_FOR_HUMAN_REVIEW**. This is independent ELEC5623 Track B coursework.

First-month scope is Python 3.11, FastAPI, Pydantic v2, scikit-learn, and pytest, with synthetic English JSON/Markdown only. Live commercial compare APIs are out of scope this week. Team allocation, stakeholder evidence, tutor approval of this Workbench direction, the annotation protocol, final thresholds, and frozen-corpus results remain open gates.

3. Introduction and background

The project addresses AI engineering evaluation: whether generated output uses a bounded evidence set and satisfies stated requirements. Whole-document scores can hide mixed support. FActScore motivates decomposing long-form generation into atomic facts because supported and unsupported content may coexist [4]. This proposal adopts that claim-level motivation only; it does not reuse the FActScore implementation, data, experiments, or metric. RAGAs separates aspects such as retrieval relevance and faithful use of retrieved information [5]. The Inspector similarly keeps retrieval matches, support labels, evidence-integrity checks, requirement mappings, and human decisions visible instead of collapsing them into one unexplained score. Token spend and model list price are not treated as quality or task-fit proxies. It does not implement the RAGAs suite.

The assumed current workflow is manual: identify claims, search supplied evidence, check citations and quotations, compare requirements, and record a decision. This workflow and its cost are hypotheses; no interview, time saving, error reduction, organisation, or market figure is claimed. A permitted stakeholder method or tutor-approved alternative must establish which review step is costly, which errors matter most, and which trace fields users need.

Alternative solution classes include manual review, document search, generic LLM critique prompts, research evaluation frameworks, and broad governance platforms. Manual review preserves authority but may vary in structure. Document search retrieves passages without assessing claims. Generic prompts are convenient but do not guarantee stable schemas, immutable inputs, or fail-closed behaviour. Existing research tools provide credible context but target different data, metrics, or workflows. The proposed contribution is a narrow, independent course product joining claim evidence, requirements, metrics, and human review in one reproducible contract.

4. Problem statement and motivation

The target decision is: **given the same stated requirements, supplied evidence, and outputs from at least two named models, which model hallucinates less / fits the task better, and should min-cost or quality/task-fit drive that choice?** The gap is not another generator. It is a bounded packet that links input hash, stable claim, evidence, support decision, mapped requirements, per-model quality/task-fit/time/estimated-cost, two named selections, and later human review.

Students checking a lab write-up risk missing invented numbers; reviewers risk overlooking weak claims; leads need reproducibility. These remain hypotheses until validated. GenAI is appropriate only for interpreting varied claim-evidence language. Deterministic code owns schema, ranking, citation and quote integrity, artifacts, and approval boundaries. Estimated cost is list-price $\times$ conservative token bound, not an invoice. The product does not auto-deploy a model and does not claim that the quality rule predicts deployment success. No unverified time, financial, accuracy, or market claim is made.

5. Stakeholders and requirements engineering

5.1 Stakeholder needs and validation

Hypothesised stakeholder	Need	Main design risk
Evidence reviewer	Locate support and challenge weak claims in one queue	Automation bias or missed context
Requirement owner	See supported claim-level coverage and explicit gaps	Lexical mapping overstates semantic coverage
Report author	Correct unsupported, contradicted, or misquoted statements	Writing to the tool rather than to evidence
Team lead or assessor	Reproduce inputs, configuration, output, and review history	Synthetic metrics create false confidence
System administrator	Operate a bounded, fail-closed evaluation path	Data leakage or provider outage

The proposed primary stakeholders are evidence reviewers, requirement owners, and report authors. Proposed secondary stakeholders are team leads or assessors and system administrators. This classification is a hypothesis that the group and tutor must confirm rather than an established organisational fact.

Validation will ask which step consumes most attention, whether missed risky claims or false escalations are costlier, and what evidence a decision record must retain. The group will record the permitted method, consent, findings, and resulting requirement changes. Identifiable notes stay private. If direct stakeholder contact is not permitted or feasible, **[REQUIRED: record the tutor-approved validation alternative]**; no participant or result will be invented.

5.2 Scope and priority

Must scope includes strict schemas and IDs, stable claim segmentation, local retrieval, five support labels, citation and exact-quote enforcement, requirement mapping and metrics, no-clobber artifacts, append-only review, CLI/API parity, sequential named-model compare with min-cost vs quality/task-fit, and fail-closed tests. Should scope includes frozen-set error analysis and clean-checkout rehearsal. Could scope includes lightweight report visualisation or a second synthetic domain. The first month will not include PDF/image/OCR input, web search, a database, a frontend framework, live commercial compare APIs, autonomous approval, or cross-project assessed artifacts.

5.3 Functional requirements

All 18 FRs are draft until the group and tutor confirm scope and allocation. The official minimum is three FRs per person. Eighteen FRs still allow at least three per person for a confirmed team of no more than five.

ID	Functional requirement	Testable acceptance criterion
FR-01	Validate a versioned English JSON bundle with strict schemas.	Invalid or extra fields fail before a run.
FR-02	Reject duplicate requirement, evidence, and expected-claim IDs.	Each duplicate category fails with no report.
FR-03	Segment English Markdown prose/lists into stable claim IDs.	Same input yields the same ordered IDs; heading/code-only fails.
FR-04	Retrieve ranked local evidence for every claim.	Each claim records bounded matches, scores, and excerpts.
FR-05	Use TF-IDF and identify any lexical fallback.	Each run records <code>sklearn-tfidf</code> or the named fallback.
FR-06	Produce exactly one of five support labels per claim.	Output is one of the five-label enum values.
FR-07	Extract evidence citations for each claim.	Citation IDs are explicit and traceable to the bundle.
FR-08	Block positive support based on an unknown citation.	Unknown citations cannot remain positive.
FR-09	Check quoted spans exactly against cited evidence.	An altered quote cannot retain positive support.
FR-10	Map claims only to stated requirement IDs.	Unknown IDs are rejected; mappings stay in-bundle.
FR-11	Calculate citation, quote, coverage, and label metrics.	JSON and Markdown exports agree.
FR-12	Score only complete supplied truth.	Non-empty <code>expected_claims</code> must cover segmented IDs exactly.
FR-13	Export/read no-clobber artifacts.	Aliased, partial, or mismatched reports fail.
FR-14	Append reviews without rewriting evidence.	History/hash persist; alias targets fail unchanged.
FR-15	Expose one Inspector service through CLI and HTTP.	CLI and API contract tests pass.
FR-16	Sequentially run ≥ 2 named gateways on the same bundle.	Each model yields an independent Inspector run.
FR-17	Report quality, task-fit, time, estimated cost, and two selections.	Min-cost vs quality/task-fit; no combined index.
FR-18	If complete labels exist, check each policy vs annotation-preferred.	Same ranking keys as quality/task-fit; else comparison only.

The public CLI is `validate bundle.json`, `evaluate bundle.json --out runs/`, `compare bundle.json --out DIR --models fixture,fixture-b`, and `review RUN_ID review.json --runs runs/`. The HTTP surface is `GET /health`, `POST /v1/evaluations`, `GET /v1/evaluations/{run_id}`, and `POST /v1/evaluations/{run_id}/reviews`. Adapters call the same Inspector service rather than duplicating decision rules.

5.4 Non-functional requirements, constraints, and assumptions

ID	Quality and proposed measurable criterion
NFR-01	Traceability: 100% of serialized assessments contain required claim, evidence, decision, requirement, configuration, and input-hash fields.
NFR-02	Reproducibility: two fixture runs produce byte-equivalent normalized reports.
NFR-03	Performance: 200 frozen claims complete in ≤ 60.0 seconds on the nominated lab machine.
NFR-04	Token budget: a conservative canonical UTF-8 byte bound is checked before provider use.
NFR-05	Fail closed: unsafe input, incomplete/mismatched truth, or provider failure creates no completed <code>report.json</code> .
NFR-06	Privacy: only the minimum configured payload crosses an approved provider boundary.
NFR-07	Auditability: runs are no-clobber; report/review targets are unaliased; reviews are append-only.
NFR-08	Onboarding: a new member reproduces the README flow within 20 minutes.
NFR-09	Maintainability: core package branch coverage remains $\geq 80\%$.
NFR-10	Comparison latency: two-model fixture compare of the sample/daily bundle finishes in ≤ 60.0 s; a failed model publishes no complete compare report.

The 60-second, budget, onboarding, and coverage values are proposed engineering criteria, not marking thresholds. They remain subject to tutor feedback and measurement on the nominated environment.

ID	Design constraint and consequence
C-01	Python ≥ 3.11 , < 3.12 and fixed first-month libraries provide one bounded toolchain.
C-02	English JSON/Markdown only; no PDF, image, OCR, or multilingual parsing.
C-03	Supplied local evidence only; no web retrieval or database.
C-04	<code>FixtureModelGateway</code> is CI truth; live-model output cannot replace reproducibility.
C-05	Synthetic data only until provenance, privacy, licence, and tutor approval are recorded.
C-06	Every successful machine run stops at human review; no approval or external action.

Assumptions requiring validation are that users can supply a bounded bundle, English text is sufficient for the first milestone, the course environment can run Python 3.11, and a synthetic corpus is acceptable. A failed assumption triggers recorded scope review rather than silent expansion.

5.5 Requirement-to-evaluation trace

Stakeholder need	Priority	Linked requirements	Candidate acceptance evidence
Review a claim with bounded evidence and no hidden approval	Must	FR-04, FR-06-09, FR-13-14; NFR-01, NFR-07	Claim trace contains ranked evidence, enforced citation/quote status, immutable report, and appended human decision.
Identify supported requirement coverage and explicit gaps	Must	FR-10-12; NFR-01	Mapping remains within stated IDs and frozen truth yields per-requirement precision, recall, and macro-F1.
Correct risky generated statements	Must	FR-03, FR-06, FR-08-09	Stable claims expose unsupported, contradicted, unknown-citation, and altered-quote cases with reasons.
Reproduce a review packet	Must	FR-13, FR-15; NFR-02, NFR-08	Clean fixture run reproduces normalized artifacts through the documented CLI/API path.
Compare named models without a hidden score	Must	FR-16-18; NFR-10	Two policies and optional label match; fail-closed incomplete compares write no <code>compare.json</code> .
Operate within budget, privacy, and failure boundaries	Must	FR-01-02, FR-15; NFR-04-06	Invalid, unsafe, over-budget, or provider-failed requests create no completed report and disclose no unapproved payload.

6. Proposed product and innovation

The Workbench is a specialised evaluation pipeline, not a chatbot. A user validates one input bundle, evaluates named models, inspects retained reports, compares min-cost versus quality/task-fit, and appends a human review. The Inspector engine supplies claim-evidence audits; the compare shell does not replace it.

The AI component is deliberately replaceable. A **ModelGateway** interprets claim-evidence relationships; deterministic components control validation, budget and injection preflight, segmentation, retrieval provenance, evidence-integrity enforcement, metrics, artifacts, and review history. A provider response therefore cannot authorise an action or silently bypass evidence rules.

The engineering contribution is the observable linkage **claim -> evidence -> support decision -> requirement -> human review**. Unlike a generic critique prompt, the product has versioned schemas, stable claim IDs, retained citations and input hashes, bounded requirement IDs, typed model output, deterministic post-checks, no-clobber history, and explicit failure tests. It is independent from FActScore and RAGAs beyond cited context [4], [5]. This ELEC5623 product is independent new work; see **PRIOR_WORK_DISCLOSURE.md**. Tutor approval of that boundary is still pending.

7. Proposed methodology and system design

Development uses requirement-led vertical slices. For each approved requirement, the group defines an acceptance criterion, implements behind the shared service, adds positive and negative pytest tests [7], retains an evidence handle, and updates traceability. Scope changes are checked against Must/Should/Could/Won't priorities and require the recorded tutor agreement where they affect the approved topic or core requirements.

English JSON bundle

- > Pydantic schema and uniqueness validation
- > token-budget and prompt-injection preflight
- > deterministic Markdown claim segmentation
- > local evidence retrieval (TF-IDF or named lexical fallback)
- > ModelGateway support classification
- > citation and exact-quote enforcement
- > requirement mapping and metrics
- > no-clobber input/report artifacts
- > append-only human review

First-month data comes only from newly created synthetic English JSON bundles and AI-generated Markdown stored on the local filesystem. CI uses the deterministic **FixtureModelGateway**; an Azure/OpenAI-compatible gateway is an optional integration only after model, budget, privacy, account, and tutor conditions are recorded. The local filesystem remains the first-month artifact store; no database or web retrieval is introduced.

Pydantic rejects unknown or invalid fields. Preflight applies a conservative canonical UTF-8 bound and a tested injection guard. Segmentation excludes headings and code fences and rejects output without auditable prose. Retrieval records its backend; scikit-learn documents the TF-IDF feature conversion [6], while local tests define this product's behaviour. Fixture mode is deterministic. An optional OpenAI-compatible gateway remains disabled until endpoint, deployment, model settings, budget, privacy, and tutor conditions are recorded. External credentials require HTTPS because authentication and provider policy demand it.

After model classification, deterministic rules force unknown citations, uncited positive decisions, and false quotations to **UNSUPPORTED**. Each successful evaluation reserves a distinct directory, writes canonical input and human-readable output, and writes **report.json** last as the completion marker. A partial write cannot be retrieved as success. Reviews append to **reviews.jsonl** without modifying the report.

Principal limitations are lexical retrieval, English-only segmentation, exact-string quote checking, synthetic data, no authentication layer, local filesystem assumptions, and no evidence that fixture results generalise to natural documents. These limitations are evaluation targets, not hidden implementation details.

8. Business and product analysis

The value proposition is a **reproducible human-review packet and a named-model comparison, not auto model-deploy**. Candidate users gain a stable queue in which evidence, requirements, machine rationale, and later human decisions remain inspectable, and in which quality, task fit, latency, and estimated list-price cost are reported separately. Min-cost and quality/task-fit are contrasted, never collapsed. These benefits remain stakeholder hypotheses.

Alternative	Strength	Gap for this proposed need	Workbench response
Manual review	Human context and authority	Structure and retained evidence may vary	Preserve authority while standardising the packet
Document search	Fast passage lookup	No support classification or requirement map	Add claim-level audit and bounded mapping
Generic LLM critique	Low setup effort	Free-form, unstable, and may invent evidence	Strict schemas and deterministic enforcement
Research framework	Established concepts	Different task, data, metric, or workflow	Cite context; implement independent requirements
Broad governance platform	Operational breadth	Potentially opaque, inaccessible, or too broad	Deliver a narrow transparent semester slice
Token or model-price comparison	Easy to collect	Silent on quality and task fit; can mislead selection	Keep dimensions separate; two named policies, not a hidden index

Technical feasibility is supported by the existing Python vertical slice, contract tests, and deterministic fixture path, but final feasibility depends on the approved scope and course environment. Team feasibility is unknown until members, availability, skills, and FR allocation are confirmed. The main costs are engineering and review time, annotation effort, approved model usage if enabled, CI/runtime resources, and retained artifacts. No currency, cloud credit, customer, price, market-share, or savings claim is made. This is independent ELEC5623 Track B coursework.

Operational needs include Python onboarding, secrets outside source and artifacts, HTTPS for external credentials, storage retention, model-version records, provider outage handling, and ownership of the human queue. Authentication, tenancy, production support, and a commercial launch are out of scope. Before submission the group must validate the target workflow, error trade-off, required trace fields, and willingness to review, or record a tutor-approved alternative.

9. Evaluation plan

The evaluation asks whether the Inspector preserves complete claim-evidence traces (RQ1), distinguishes five support labels (RQ2, quality), detects unsupported and contradicted claims (RQ3, quality), maps stated requirements (RQ4, task fit), fails closed (RQ5, efficiency), can be reproduced (RQ6), and whether min-cost vs quality/task-fit agree with annotation-preferred when labels exist (RQ7, Workbench). Dimensions are not combined into one score. Without labels, compare is comparison only and must not claim prediction or success. Estimated cost is list-price × conservative token bound, not a bill.

The planned dataset is 20 bundles and 200 claims, containing only newly created synthetic requirements, evidence, Markdown, expected labels, and requirement mappings for ELEC5623. The corpus is independent of prior personal or other-unit work. Before final evaluation, the group must obtain tutor approval, freeze operational label and ambiguity rules without seeing final predictions, use two permitted annotators or an approved alternative, resolve disagreement independently of model output, hash the corpus and labels, and prohibit tuning on the final labels. The current generator and fixture are regression evidence, not independent annotation or a frozen result.

For each class, precision is $TP/(TP+FP)$, recall is $TP/(TP+FN)$, and F1 is their harmonic mean. Five-label macro-F1 is the unweighted mean of the five class F1 values. Risky precision and recall treat **UNSUPPORTED** and **CONTRADICTED** as the combined positive set. Requirement-mapping macro-F1 is calculated per requirement then averaged. Citation completeness is the proportion of claims with at least one citation for which every cited ID exists.

Measure	Proposed target
Citation completeness	1.00
Five-label macro-F1	≥ 0.65
Risky precision	≥ 0.80
Risky recall	≥ 0.75
Requirement-mapping macro-F1	≥ 0.70
Elapsed time for 200 claims	≤ 60.0 seconds on the nominated machine
Two-model sample/daily compare	≤ 60.0 seconds; incomplete compares publish nothing

These values are engineering proposals, not course thresholds. Tutor feedback, stakeholder trade-offs, and a development-set rationale must confirm or revise them before freezing the evaluation.

Comparison	Planned control
B0 trivial constant or frequency baseline	Fit only on development data; report even if weak
B1 TF-IDF, fixture gateway, deterministic enforcement	Reproducible Inspector baseline; list-price cost 0
B1b gold-label fixture-b	Workbench contrast with published mock list price > 0
B2 one compatible model	Out of compare CLI this week; privacy/course approval later

Same-input ablations will remove TF-IDF, deterministic evidence enforcement, or the normal requirement-mapping signal. Failure tests cover malformed schema, duplicate or unknown IDs, missing or ghost ground-truth claim IDs, heading/code-only output, prompt injection, budget excess, provider timeout or invalid JSON, unknown citations, altered quotes, partial and reused artifacts, aliased report/review targets, and append-only review. Every unsafe failure must produce no completed report or outside-target change. Clean-checkout tests will reproduce the fixture run and verify normalized byte equivalence; timed performance will be measured on the nominated lab machine with runtime, hardware load, gateway, and command recorded.

The present local vertical slice has 152 passing tests and 94.02% branch coverage. The current no-clobber acceptance at [acceptance/local-20260804-review-integrity-v2/corpus-acceptance.json](#) reports 20 bundles and 200 claims: macro-F1 0.857701, risky precision/recall 0.930233/1.0, mapping macro-F1 0.96, and 37 label plus 12 mapping errors. Its status remains **DRAFT_UNFROZEN_NOT_TUTOR_APPROVED**; these local fixture results are not final acceptance evidence. The submission must report a later frozen result separately with per-class metrics, errors, hashes, and all failed thresholds.

If permitted, at least one reviewer who did not author the evaluated bundle will locate a risky claim, inspect evidence and requirement traces, append a decision, and judge packet sufficiency. Record task completion, correctness against frozen annotation, time, missing fields, and a short usefulness/trust rating while keeping identity and comments private. If unavailable, use the tutor-approved qualitative alternative. Validity threats include lexical and exact-quote proxies (construct), generator-fixture alignment (internal), 20 English synthetic bundles (external), aggregate-only interpretation (conclusion), and environment drift (reproducibility).

9.1 Evaluation trace and interpretation rules

Research question	Primary evidence	Candidate decision rule	Required report content
RQ1: Are claim-evidence traces complete?	Schema checks, citation completeness, manual packet sample	Target 1.00; any missing required trace field is a failure, not a partial pass.	Missing fields by claim and producing stage
RQ2: Are five support labels distinguished?	Frozen confusion matrix and per-class metrics	Proposed macro-F1 ≥ 0.65 ; also report every class even if the aggregate passes.	Per-class precision/recall/F1 and confusion matrix
RQ3: Are risky claims surfaced?	Combined unsupported/contradicted precision and recall	Proposed precision ≥ 0.80 and recall ≥ 0.75 ; neither metric may hide the other.	False-support and false-escalation examples
RQ4: Are requirements mapped accurately?	Frozen per-requirement truth and macro-F1	Proposed macro-F1 ≥ 0.70 with unknown IDs rejected before scoring.	Per-requirement errors and uncovered requirements
RQ5: Does the product fail closed?	Negative suite for injection, budget, provider, evidence, and artifact faults	Every unsafe case produces no completed report; any readable partial success fails.	Input, expected failure, observed artifact state, and diagnostic
RQ6: Can the result be reproduced and reviewed?	Byte-equivalent fixture runs, clean checkout, and permitted reviewer task	Exact fixture equivalence plus documented environment; qualitative findings remain separate from quantitative claims.	Runtime/configuration, hashes, task outcome, time, and missing trace fields
RQ7: Do named policies agree with labels?	Two-model compare on a labelled bundle	Min-cost vs quality/task-fit vs annotation-preferred; unlabelled runs set verification unavailable.	Selected models, costs, quality metrics, and match flags

Threshold outcomes will be reported as pass or fail without suppressing failed classes, scenarios, or requirements. Error analysis will select representative false support, false escalation, mapping error, citation/quote enforcement, and provider-failure cases using a rule fixed before the final run. The group will link every reported table or figure to a retained run ID and source hash so the proposal, later report, demo, journal, and commit can be reconciled.

10. Risks, ethics and responsible AI

NIST AI RMF organises AI risk work through GOVERN, MAP, MEASURE, and MANAGE and identifies accountability and transparency as trustworthiness concerns [1]. Its GenAI Profile discusses confabulation, privacy, human-AI configuration, automation bias, information integrity, and security [2]. These references guide questions; they do not prove that this prototype is safe or compliant.

The main ethical risk is presenting a support label as truth. The interface and report therefore expose evidence and uncertainty, keep **READY_FOR_HUMAN_REVIEW** as the only successful machine state, and record human decisions separately. The demo must include a human challenge to an automated assessment. Both risky precision and recall are reported because false support may hide a weak claim while false escalation may waste effort or unfairly undermine a valid one.

Synthetic data is the default until provenance, consent, licence, retention, and approval are recorded. Secrets stay outside source and evidence. OWASP describes direct and indirect prompt injection and layered controls including output validation, least privilege, untrusted-content separation, adversarial testing, and human approval [3]. The current pattern guard is not foolproof; the stronger control is that the gateway cannot write artifacts or act externally. Bias can enter domains, labels, annotation, and per-class performance, so errors will be reported by class. Data rights, provider terms, assessment rules, queue ownership, over-reliance, and onboarding remain legal or adoption gates. No production security or regulatory compliance is claimed.

Risk	Current mitigation	Trigger and response	Owner gate
Tutor or prior-work decision conflicts with scope	Early approval pack and disclosure	Stop freeze; revise and reconfirm	[REQUIRED: authorised tutor]
Team availability differs	Conditional allocation and shared review	Replan without inventing members	[REQUIRED: actual group]
Synthetic corpus overfits fixture rules	Independent protocol and frozen labels	Report limitation; redesign evaluation	[REQUIRED: evaluation owner]
Provider leaks data or fails	Synthetic minimum payload, HTTPS, fixture CI	Fail closed; use approved fixture path	[REQUIRED: technical owner]
Automation bias	Human-only decision and challenge case	Record disagreement; revise presentation	[REQUIRED: whole group]
Requirement or schedule drift	Traceability, hashes, weekly gates	Block report/demo; cut Should/Could first	[REQUIRED: project owner]

Direct GenAI and prior-work disclosure

OpenAI Codex was materially used on 2 August 2026 to scaffold and review the independent prototype, tests, evaluation tooling, and this proposal candidate. Its outputs were revised and checked using local tests, source provenance, and separate Codex-assisted adversarial review passes; no independent human review is implied. Codex does not establish stakeholder facts, team authorship, tutor approval, frozen-corpus validity, or course compliance. The submitting group remains responsible for every claim, citation, requirement, engineering decision, and permitted-use declaration.

AI_JOURNAL.md records detailed use, but does not replace this direct acknowledgement. Before submission the group must reconcile the wording with current course and University guidance, record all later material AI use and human review, and retain the tutor's decision on **PRIOR_WORK_DISCLOSURE.md**.

11. Semester plan and team responsibilities

Period	Intended outcome and exit evidence	External dependency
3-9 Aug	Form/register group; confirm pitch, schema, 18 FRs, and prior-work disclosure	Members and Canvas group rule
10-16 Aug	Complete Lab 1 environment evidence and single-bundle vertical slice	Private approved accounts and course environment
17-23 Aug	Complete Week 3 oral check; record feedback and scope decision	All-member participation and tutor feedback
24-30 Aug	Approve corpus protocol; implement baselines, failure tests, and draft evaluation	Tutor/annotation decision
31 Aug-7 Sep	Obtain formal approval; reconcile 8-10 page PDF, citations, AI use, and all versions	Tutor, group review, live Canvas check
8 Sep 23:59	Student submits one group PDF and retains confirmation privately	Student action
9 Sep-late Sep	Implement the approved vertical slice, integrate Must interfaces, and convert new Canvas criteria into traceability	Published assessment/rubric updates
Late Sep-mid Oct	Reach the internal MVP target; close Must tests and run failure scenarios without expanding approved scope	Team integration and approved environment
Mid-late Oct	Reach the internal feature-complete target; freeze the permitted test set and produce the report/error-analysis draft	Frozen protocol and later brief
Final two teaching weeks before the published demo	Feature freeze, clean-checkout reproduction, report/AI-journal audit, and at least two demo/Q&A rehearsals	Official final-assessment instructions
Official final demonstration date once published	Demonstrate the version matched by report, journal, evidence, and final commit	Canvas date and in-person assessment

For a confirmed five-person group, the neutral allocation is Slot A FR-01-03, Slot B FR-04-06, Slot C FR-07-09, Slot D FR-10-12, and Slot E FR-13-18. For fewer members, all 18 FRs must be redistributed while preserving at least three per person. Replace slots with **[REQUIRED: actual names, SIDs, availability, primary deliverables, and reviewers]** only after agreement. Every member remains responsible for the complete proposal.

The group will log each requirement change with source, owner, and impacts on implementation, tests, evidence, and report. A feature needs matching acceptance evidence and review. Proposal, code, tests, demo, AI journal, and final commit must describe one version. Must defects outrank Should/Could features, and private course or participant data stays private.

Before rebuilding this candidate as the submission PDF, the group must close these gates: live Canvas rule recheck; group registration and allocation; Week 3 participation; written tutor approval and cover details; stakeholder or approved alternative evidence; prior-work decision; annotation and threshold decision; references and AI-use audit; final internal consistency and member sign-off. The student/group alone performs the final Canvas submission. No appendix allowance is assumed.

12. References

- [1] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, Jan. 2023, doi: [10.6028/NIST.AI.100-1](https://doi.org/10.6028/NIST.AI.100-1).
- [2] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*, NIST AI 600-1, July 2024, doi: [10.6028/NIST.AI.600-1](https://doi.org/10.6028/NIST.AI.600-1).
- [3] OWASP Gen AI Security Project, "LLM01:2025 Prompt Injection," accessed 2 Aug. 2026. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>.
- [4] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FactScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," in *Proceedings of EMNLP 2023*, pp. 12076-12100, 2023, doi: [10.18653/v1/2023.emnlp-main.741](https://doi.org/10.18653/v1/2023.emnlp-main.741).
- [5] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated Evaluation of Retrieval Augmented Generation," in *Proceedings of the 18th EACL: System Demonstrations*, pp. 150-158, 2024, doi: [10.18653/v1/2024.eacl-demo.16](https://doi.org/10.18653/v1/2024.eacl-demo.16).
- [6] scikit-learn developers, "TfidfVectorizer," *scikit-learn API Reference*, accessed 2 Aug. 2026. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.
- [7] pytest developers, "pytest documentation," accessed 2 Aug. 2026. <https://docs.pytest.org/en/stable/>.

All seven references were opened and metadata-checked on 2 August 2026. They support design context only and do not prove implementation performance, safety, compliance, stakeholder demand, tutor approval, or a frozen result.

NOT FOR SUBMISSION